University Scholarship Allocation System

Problem Statement: Design a system that evaluates students based on aggregate marks, subject-wise minimum marks, attendance percentage, and family income. The system selects eligible students, ranks them, and resolves ties using predefined rules.

1. Computational Thinking

1. Decomposition: Divide the problem into smaller tasks:

- Collect student details.
- Calculate aggregate marks.
- Check subject-wise minimum marks.
- Verify attendance and family income.
- Select eligible students.
- Rank students and resolve ties.

2. Pattern Recognition: Identify common eligibility conditions among all applicants.

3. Abstraction: Focus only on marks, attendance, income, and student details.

4. Algorithm Design: Create step-by-step instructions to select and rank eligible students.

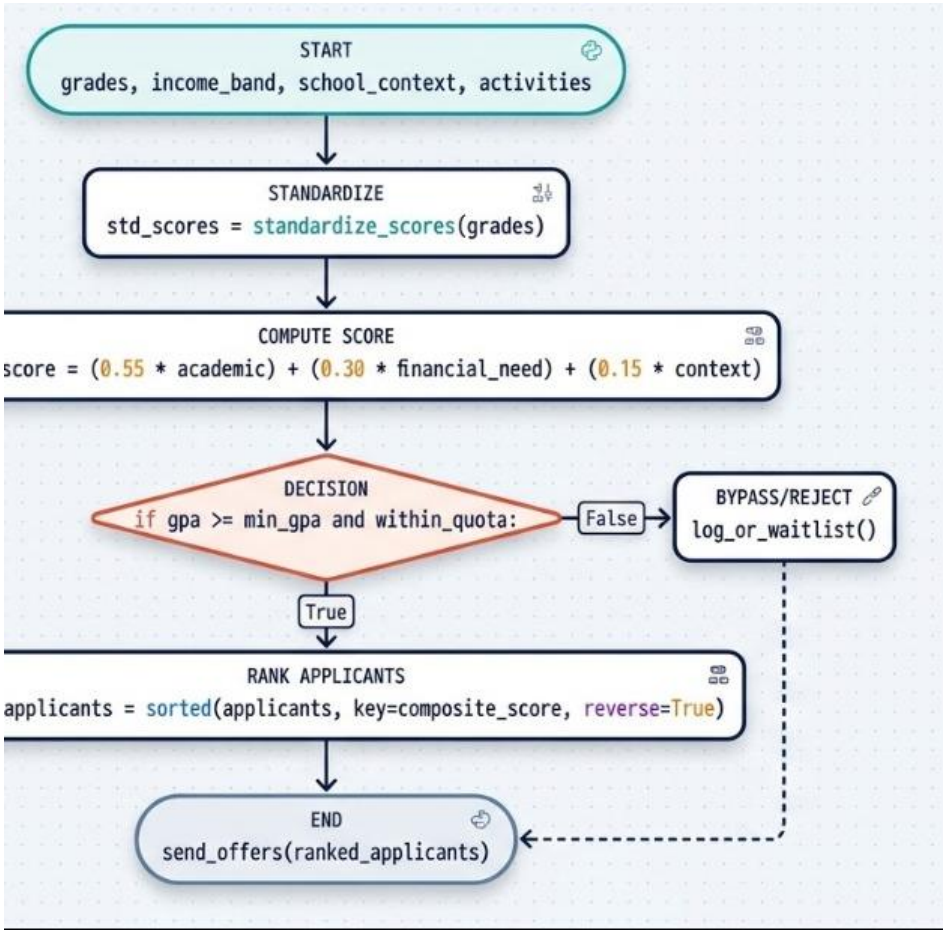
2. Algorithm

1. Start.
2. Read the number of students.
3. Read each student's name, marks, attendance, and family income.
4. Calculate aggregate percentage.
5. Check whether all subjects meet minimum marks.
6. Check attendance is at least 75%.
7. Check family income is within ₹2,50,000.
8. If all conditions are satisfied, mark the student as eligible.
9. Store eligible students in a list.
10. Sort eligible students based on aggregate percentage in descending order.
11. If aggregate marks are equal, give priority to:
    - Higher attendance.
    - Lower family income.

12. Display the final scholarship ranking.

13. Stop.

3. Flowchart



Flowchart explanation:

Start → Input student details → Calculate aggregate → Check subject minimum → Check attendance → Check income → Eligible or Not Eligible → Rank eligible students → Display result → Stop.

4. Variables

Variable	Purpose
n	Number of students
name	Student name
marks	Subject marks
total	Total marks

aggregate	Aggregate percentage
attendance	Attendance percentage
income	Family income
eligible	Eligibility status
rank	Student rank

5. Data Types

Data type	Variables
Integer	n, total, rank
Float	aggregate, attendance, income
String	name, eligibility
Boolean	eligible
List	Student records
Tuple	Student details

6. Operators

Operator	Purpose
+	Addition of marks
/	Calculate average
>=	Minimum marks and attendance
<=	Income limit
==	Compare values
and	Check all conditions
>	Compare aggregate marks

7. Conditional Statements

Use if, elif, and else.

if subject_minimum and attendance >= 75 and income <= 250000: eligible = Trueelse: eligible = False

8. Loops

Use a for loop to process multiple students.

for i in range(n): print("Enter student details")

Use loops to check each student's marks and eligibility.

9. Strings

Strings are used to store student names and eligibility messages.

name = "Ravi"status = "Eligible"print(name, status)

10. Pattern Generation

Generate a simple scholarship ranking pattern.

for i in range(1, 6): print("*" * i)

Output:

*

**

***

****

*****

11. Lists

A list stores multiple eligible students.

students = ["Ravi", "Sita", "Kiran"]print(students)

12. Tuples and Packing

A tuple stores student details together.

student = ("Ravi", 90, 85, 200000)name, marks, attendance, income = studentprint(name, marks, attendance, income)

Here, packing combines values into a tuple, and unpacking assigns them to individual variables.

13. Pseudocode

START

INPUT n

```
CREATE empty eligible_list

FOR each student from 1 to n:

    INPUT name
    INPUT subject marks
    INPUT attendance
    INPUT income

    CALCULATE aggregate

    IF all subject marks >= minimum
       AND attendance >= 75
       AND income <= 250000 THEN

          ADD student details to eligible_list

    ELSE
          PRINT "Not Eligible"

    END IF

END FOR

SORT eligible_list BY:
    1. Aggregate in descending order
    2. Attendance in descending order
    3. Income in ascending order
```

ASSIGN RANKS

DISPLAY eligible students and ranks

STOP

14. Python Program

students = []n = int(input("Enter number of students: "))for i in range(n): print("\nStudent", i + 1) name = input("Enter name: ") marks = list(map(float, input("Enter 3 subject marks: ").split())) attendance = float(input("Enter attendance: ")) income = float(input("Enter family income: ")) aggregate = sum(marks) / len(marks) if (all(m >= 40 for m in marks) and attendance >= 75 and income <= 250000): students.append((name, aggregate, attendance, income)) else: print(name, "Not Eligible")students.sort(key=lambda x: (-x[1], -x[2], x[3]))print("\nSCHOLARSHIP RANKING")for rank, student in enumerate(students, 1): print(rank, student)

15. Expected Output

SCHOLARSHIP RANKING

1 ('Ravi', 90.0, 90.0, 200000)

2 ('Sita', 85.0, 88.0, 180000)

3 ('Kiran', 80.0, 85.0, 220000)

Note: These are sample results. The program uses 40 as the minimum subject mark, 75% attendance, and ₹2,50,000 income limit.

16. Conclusion

The University Scholarship Allocation System uses computational thinking, algorithms, flowcharts, variables, data types, operators, conditional statements, loops, strings, pattern generation, lists, and tuples to identify eligible students and rank them fairly.